Assign2_bda

shreya@ShreyaYadav:~/mapreduce$ touch mapper2.py

shreya@ShreyaYadav:~/mapreduce$ touch reducer2.py

shreya@ShreyaYadav:~/mapreduce$ chmod +x mapper2.py reducer2.py

shreya@ShreyaYadav:~/mapreduce$ hadoop fs -rm -r /mat_out

rm: Call From ShreyaYadav/127.0.1.1 to localhost:9000 failed on connection exception: java.net.ConnectException: Connection refused; For more details see: http://wiki.apache.org/hadoop/ConnectionRefused

shreya@ShreyaYadav:~/mapreduce$ hadoop fs -put matrix.txt /input/

put: Call From ShreyaYadav/127.0.1.1 to localhost:9000 failed on connection exception: java.net.ConnectException: Connection refused; For more details see: http://wiki.apache.org/hadoop/ConnectionRefused

shreya@ShreyaYadav:~/mapreduce$ start-dfs.sh

Starting namenodes on [localhost]

Starting datanodes

Starting secondary namenodes [ShreyaYadav]

shreya@ShreyaYadav:~/mapreduce$ start-yarn.sh

Starting resourcemanager

Starting nodemanagers

shreya@ShreyaYadav:~/mapreduce$ jps

97409 NodeManager

96720 DataNode

96933 SecondaryNameNode

97604 Jps

96587 NameNode

97279 ResourceManager

shreya@ShreyaYadav:~/mapreduce$ chmod +x mapper2.py reducer2.py

shreya@ShreyaYadav:~/mapreduce$ hadoop fs -rm -r /mat_out

rm: `/mat_out': No such file or directory

shreya@ShreyaYadav:~/mapreduce$ hadoop fs -put matrix.txt /input/

put: `matrix.txt': No such file or directory

shreya@ShreyaYadav:~/mapreduce$ hadoop fs -put matrix.txt /input/

shreya@ShreyaYadav:~/mapreduce$ find /usr/local/hadoop -name "hadoop-streaming*.jar"

/usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar

/usr/local/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.3.6-sources.jar

/usr/local/hadoop/share/hadoop/tools/sources/hadoop-streaming-3.3.6-test-sources.jar

shreya@ShreyaYadav:~/mapreduce$ hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \

-file mapper.py -mapper "python3 mapper2.py" \

-file reducer.py -reducer "python3 reducer2.py" \

-input /input/matrix.txt \

-output /mat_out

shreya@ShreyaYadav:~/mapreduce$ hadoop fs -cat /mat_out/part-00000

Delete the failed output folder:

Bash

hadoop fs -rm -r /mat_out

mapper.py

#!/usr/bin/env python3

import sys

For a 2x2 * 2x2 multiplication:

```
# m = rows of A, p = columns of B
m = 2
p = 2

for line in sys.stdin:
    parts = line.strip().split(',')
    if len(parts) != 4: continue

    mat, i, j, val = parts[0], int(parts[1]), int(parts[2]), float(parts[3])

    if mat == "A":
        # A[i,j] is needed for every column 'k' of the result row 'i'
        for k in range(p):
            print(f"{i},{k}\tA,{j},{val}")
    else:
        # B[i,j] is needed for every row 'k' of the result column 'j'
        for k in range(m):
            print(f"{k},{j}\tB,{i},{val}")
```

reducer.py

```
#!/usr/bin/env python3
import sys

current_cell = None
data = []
```

```
for line in sys.stdin:
    cell, val = line.strip().split('\t')

    if current_cell == cell:
        data.append(val)
    else:
        if current_cell:
            # Calculate the cell value
            dict_a = {}
            dict_b = {}
            for item in data:
                name, index, value = item.split(',')
                if name == "A": dict_a[index] = float(value)
                else: dict_b[index] = float(value)

            # Multiply matching indices and sum
            total = 0
            for k in dict_a:
                if k in dict_b:
                    total += dict_a[k] * dict_b[k]
            print(f"Cell {current_cell}\t{total}")

        current_cell = cell
        data = [val]

# Don't forget the last cell!
```

```
if current_cell:
    dict_a = {}
    dict_b = {}
    for item in data:
        name, index, value = item.split(',')
        if name == "A": dict_a[index] = float(value)
        else: dict_b[index] = float(value)
    total = sum(dict_a[k] * dict_b.get(k, 0) for k in dict_a)
    print(f"Cell {current_cell}\t{total}")
```

matrix.txt

```
A,0,0,1
A,0,1,2
A,1,0,3
A,1,1,4
B,0,0,5
B,0,1,6
B,1,0,7
B,1,1,8
```

4. Exam Execution Steps (The "Quick List")

START start-dfs.sh, start-yarn.sh

1. **Permissions:** chmod +x mapper.py reducer.py
2. **HDFS Prep:** hadoop fs -rm -r /mat_out (Delete old output) hadoop fs -put matrix.txt /input/
3. **Run:**

Bash

```
hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-3.3.6.jar \
-file mapper.py -mapper "python3 mapper.py" \
-file reducer.py -reducer "python3 reducer.py" \
-input /input/matrix.txt \
-output /mat_out
```

4. **Check Result:** hadoop fs -cat /mat_out/part-00000